

Module 03: Perception — State Estimation via Variational Inference

Learning Objectives

1. Implement belief updating using the A-matrix likelihood and fixed-point iteration.
2. Use `run_state_inference()` to infer hidden states from observations.
3. Visualize posterior beliefs and convergence diagnostics.

Introduction

Perception in Active Inference is the process of updating the agent's beliefs about hidden states given a new observation. This is not passive pattern matching — it is an active *inference* process that minimizes Variational Free Energy (VFE) by finding the posterior distribution $q(s)$ that best reconciles the observation with the agent's prior beliefs.

Key Concepts

1. The Inference Problem

Given:

- Prior beliefs: $q(s)$ (initially D , or the previous posterior)
- New observation: o_t (an integer index)
- Likelihood model: $\mathbf{A}$ where $A[o, s] = P(o \mid s)$

Find the posterior $q(s \mid o_t)$ that minimizes VFE:

$$F = D_{\text{KL}}[q(s) \mid p(s)] - \mathbb{E}_{q(s)}[\ln P(o_t \mid s)]$$

2. Fixed-Point Iteration

The `run_state_inference()` function solves this via iterative message passing:

$$q(s) \propto P(o_t | s) \cdot q_{\text{prior}}(s)$$

At each iteration:

1. Compute the log-posterior: $\ln q(s) = \ln A[o_t, :] + \ln q_{\text{prior}}(s)$
2. Normalize via softmax: $q(s) = \frac{\exp(\ln q(s))}{\sum \exp(\ln q(s))}$
3. Check convergence: $\delta = |q^{(k)} - q^{(k-1)}|$

```
from active_inference.math import run_state_inference

result = run_state_inference(
    prior=np.array([0.5, 0.5]),    # uniform prior
    observation=0,                 # observed o=0
    A=model.A,                    # likelihood matrix
    num_iterations=16,             # max iterations
    convergence_threshold=1e-8,    # stop when delta < threshold
)

print(result["q_s"])              # posterior beliefs
print(result["converged"])        # True/False
print(result["num_iters"])        # iterations used
print(result["delta_history"])    # convergence trace
```

3. How the A-Matrix Shapes Perception

The A-matrix determines how informative each observation is:

A-matrix structure	Perceptual effect
Identity ($A = I$)	Fully observable — each observation uniquely identifies a state
Uniform columns	Observations carry no information — beliefs don't update
High diagonal	Clear signal — beliefs shift strongly toward the matching state
Ambiguous (similar columns)	Weak evidence — beliefs change slowly, need more observations

```
# High-information A: observation 0 strongly implies state 0
A_clear = np.array([[0.95, 0.05],
                    [0.05, 0.95]])

# Low-information A: observations are nearly uninformative
A_noisy = np.array([[0.55, 0.45],
                    [0.45, 0.55]])
```

4. Agent-Level Inference

The `ActiveInferenceAgent.infer_states(obs)` method wraps `run_state_inference()`:

```
from active_inference.agent import ActiveInferenceAgent

agent = ActiveInferenceAgent(model, gamma=4.0)
agent.infer_states(0)          # observe o=0, update beliefs
print(agent.q_s)              # posterior
print(agent.history["vfe"])    # VFE logged after each inference
```

After inference, `agent.q_s` holds the updated posterior and VFE is appended to the history.

5. The Log-Likelihood Vector

`model.log_likelihood(obs)` returns the vector $\ln A[o, :]$ — the log-evidence that each state could have produced the observation:

```
ll = model.log_likelihood(0)    # shape: (num_states,)
# For A = [[0.9, 0.1], [0.1, 0.9]]:
# ll ≈ [-0.105, -2.303] (state 0 is much more likely to produce obs 0)
```

6. Prediction Errors

After updating beliefs, the agent can compute prediction errors:

$$\epsilon_o = \mathbf{e}_o - \mathbf{A} \cdot \mathbf{q}(s)$$

where $\mathbf{e}_o$ is a one-hot vector for the actual observation:

```
agent.infer_states(0)
pe = agent.prediction_error(0)    # shape: (num_obs,)
print(pe.sum())                  # ≈ 0 (prediction errors sum to zero)
```

7. Convergence Diagnostics

Use `plot_convergence()` to visualize how quickly inference settles:

```
from active_inference.visualization import plot_convergence

result = run_state_inference(prior=model.D, observation=0, A=model.A)
plot_convergence(result["delta_history"], threshold=1e-8)
```

Applications

- **Ambiguous stimuli:** With a noisy A-matrix, a single observation may not resolve the state — the agent needs multiple observations to gain confidence (like viewing a blurry image).
- **Bayesian surprise:** The magnitude of the belief shift after inference measures how surprising the observation was.
- **Prior-observation conflict:** When the prior strongly favors one state but the observation favors another, the posterior is a compromise weighted by the relative strengths.

Conclusion

Perception reduces to variational inference over hidden states. The A-matrix, prior beliefs, and observation jointly determine the posterior through iterated message passing. Module 04 extends this by introducing preferences (C), priors (D), and habits (E) as constraints on the agent's internal model.